

Assignment: Solving the K-Armed Bandit Problem Using Reinforcement Learning

Course

Reinforcement Learning

Assignment Title

Implementation and Evaluation of K-Armed Bandit Algorithms

Objective

The objective of this assignment is to develop a K-Armed Bandit simulation environment and implement reinforcement learning algorithms to solve the exploration-exploitation tradeoff problem. Students will gain hands-on experience with action-value estimation, greedy action selection, ϵ -greedy exploration, and performance evaluation.

Background

The K-Armed Bandit problem is one of the simplest reinforcement learning problems. An agent repeatedly selects one of K actions (arms). Each arm generates rewards according to an unknown probability distribution. The goal is to maximize the cumulative reward over time by balancing:

- **Exploration:** Trying different actions to learn their reward distributions.
 - **Exploitation:** Choosing the action currently believed to yield the highest reward.
-

Problem Description

Consider a K-Armed Bandit environment with:

- Number of arms: ($K = 10$)
- Reward distribution for each arm:

$$R(a) \sim \mathcal{N}(q_*(a), 1)$$

where:

- $(q^*(a))$ is the true action value
- $(q^*(a))$ is sampled from:

$$q_*(a) \sim \mathcal{N}(0, 1)$$

At each time step:

1. The agent selects an action.
 2. The environment returns a stochastic reward.
 3. The agent updates its action-value estimates.
-

Tasks

Part 1: Environment Development (20 points)

Develop a Python class named KArmedBandit.

The class should:

1. Initialize K arms.
2. Randomly generate true action values $(q^*(a))$.
3. Return rewards sampled from the corresponding reward distribution.
4. Identify the optimal action.

Required methods:

```
class KArmedBandit:
    def __init__(self, k=10):
        pass

    def step(self, action):
        pass

    def optimal_action(self):
        pass
```

Part 2: Greedy Action-Value Method (20 points)

Implement a Greedy Bandit Agent.

Requirements:

1. Initialize all action-value estimates to zero.
2. Select the action with the highest estimated value.
3. Update estimates using the sample-average method:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{N(a)} [R_n - Q_n(a)]$$

where:

- $Q_n(a)$: current estimate
- $N(a)$: number of times action (a) has been selected

Evaluate performance over:

- 1,000 time steps
 - 200 independent runs
-

Part 3: ϵ -Greedy Agent (25 points)

Implement an ϵ -Greedy Agent.

Evaluate the following values:

- $\epsilon = 0.0$
- $\epsilon = 0.01$
- $\epsilon = 0.10$

At each decision step:

- With probability ϵ , select a random action.
- Otherwise, select the greedy action.

Run experiments for:

- 1,000 steps
 - 200 independent runs
-

Part 4: Performance Evaluation (20 points)

Compute and plot:

Average Reward vs. Time Step

For each ϵ value:

$$\bar{R}(t) = \frac{1}{M} \sum_{i=1}^M R_i(t)$$

Percentage of Optimal Action Selection

$$\% \text{Optimal}(t) = \frac{1}{M} \sum_{i=1}^M \mathbb{I}\{A_i(t) = a^*\} \times 100\%$$

Generate the following plots:

1. Average Reward vs Time Step
2. Optimal Action Percentage vs Time Step

Include proper labels, legends, and titles.

Part 5: Analysis (15 points)

Write a short report (2–3 pages) addressing:

1. How exploration affects learning performance.
 2. Comparison of ϵ values.
 3. Why purely greedy methods may perform poorly.
 4. Tradeoffs between exploration and exploitation.
 5. Which ϵ value performed best and why.
-

Deliverables

Submit the following:

Source Code

- bandit.py
- agents.py
- experiment.py

Report (PDF)

Include:

- Problem formulation
 - Methodology
 - Experimental setup
 - Plots
 - Discussion
 - Conclusions
-

Grading Rubric

Component	Points
Environment	20
Implementation	
Greedy Agent	20
ϵ -Greedy Agent	25
Experimental Evaluation	20
Analysis and Report	15
Total	100

Expected Learning Outcomes

Upon completion of this assignment, students will be able to:

- Formulate reinforcement learning problems.
- Implement K-Armed Bandit environments.
- Develop action-value learning algorithms.
- Evaluate exploration-exploitation strategies.
- Analyze reinforcement learning performance experimentally.